



Metodología de la Programación

Grado en Ingeniería Informática

Curso 2024/2025

Rodríguez, Francisco

Gómez, Jose

Gonzalez, Daniel

Pacheco, Alfonso

Proyecto Modularidad

Hundir la flota

Índice general

Índice general	1
1. Introducción	2
2. Documentación de usuario	4
2.1. Descripción funcional	4
2.2. Tecnología	5
2.3. Manual de instalación	6
2.4. Acceso al sistema	7
2.5. Manual de referencia y guía del operador	7
3. Documentación de sistema	10
3.1. Especificación del sistema	10
3.2. Módulos	11
3.3. Plan de prueba	13
4. Documentación del código fuente	28

1. Introducción

Este es un ejemplo de plantilla para la generación de la documentación asociada al desarrollo de un programa. Se puede usar como guía, pero puede sufrir todas las modificaciones que considere oportunas para mejorar su documento.

Es necesario incluir una portada, dispone de cuatro ejemplos. Puede comprobar el resultado de cada uno de ellos en los ficheros PDF. En el fichero `.tex` está por defecto el ejemplo de Portada1.

Se debe añadir un índice. Puede observar en el código cómo se puede incluir en la sección destinada a tal fin.

A lo largo del documento también puede surgir la necesidad de añadir una imagen. Puede ver cómo hacerlo en el código de las portadas.

$\text{\LaTeX}$ también le permite incluir código. Fíjese en el código fuente cómo está delimitado. El resultado para el típico ejemplo *Hola mundo* en C es el siguiente:

```
/* Programa: Hola mundo */

#include <stdio.h>

int main()
{
    printf( "Hola mundo." );
    system("PAUSE");
    return 0;
}
```

Puede consultar tutoriales, libros o blogs para profundizar más en el aprendizaje de $\text{\LaTeX}$, resulta muy interesante por la gran ventaja que ofrece de proporcionar documentos con una alta calidad. Sea cual el sistema operativo tenga en cuenta que va a necesitar:

1. Editor de texto para generar el fichero `.tex` con el código
2. Distribución de $\text{\LaTeX}$ para compilar el fichero `.tex` y generar el fichero `.pdf`
3. Visor de documentos para poder visualizar el fichero `.pdf` generado

Las siguientes secciones indican cada una de las partes que se debe incluir en la documentación del programa. Hay que tener en cuenta que se trata de una tarea tediosa pero muy necesaria para realizar un buen mantenimiento

del programa. Para cualquier modificación del código hay que entender el mismo, siendo de gran ayuda la documentación.

Recuerde que en la documentación del programa se distinguen dos partes claramente diferenciadas:

- Documentación interna. Documentación contenida en las líneas de comentarios del programa.
- Documentación externa. Documentación que incluye el análisis, diagramas de flujos y/o pseudocódigos, manuales de usuario con instrucciones para ejecutar el programa e interpretar los resultados. Dentro de esta se distingue a su vez:
 - Documentación del usuario
 - Documentación del sistema

En las siguientes secciones se describirá el contenido a incluir en cada uno de estos tipos de documentación. El objetivo que se debe marcar en la redacción de las mismas debe ser el de conseguir un texto claro, completo y conciso, utilizando para ello construcciones gramaticales y ortográficas correctas.

Esta documentación trata del juego hundir la flota, esta es una breve introducción de lo que va a tratar este documento, e ira indicada en el propio índice: primero veremos toda la documentación necesario que necesitará un usuario para poder jugar y hacer uso con total libertad. Luego toda la documentación del sistema necesaria para poder entender todos los sistemas, osea la modularidad junto al gran rango de funciones que posee.

2. Documentación de usuario

Documentación relacionada con las funciones del sistema pero no se hace referencia a nada relativo a su implementación. Está orientado a las personas que van a usar el sistema no a los que lo van a mantener.

2.1. Descripción funcional

- ¿Qué es el juego Hundir la flota?

Hundir la flota es un juego de estrategia para dos jugadores. Cada uno coloca en secreto sus barcos en un tablero. Por turnos, intentan adivinar las posiciones del rival lanzando disparos (coordenadas). El objetivo es hundir todos los barcos del oponente antes de que él hunda los tuyos, ganará quien destruya primero toda la flota enemiga.

- ¿Cómo funciona el juego?

- ¿Qué realiza el sistema?

1. Colocar correctamente los barcos, el sistema debe verificar que los barcos no se solapen ni se salgan del tablero.
2. Dimensiones correctas para el tablero, el tablero debe ser de dimensiones válidas para la colocación de barcos.
3. Algoritmo de disparo automático, se debe implementar un algoritmo que simule disparos automáticos, ya sea aleatorio o con lógica estratégica.
4. Evaluar el disparo correctamente, el sistema deberá de evaluar según donde se realiza el disparo su consecuencia.
5. Declarar un ganador tras haber acabado la partida.

- ¿Qué no realiza el sistema?

1. Solo pueden jugar un máximo de dos jugadores.
2. Opción de elegir en el propio juego todos los tipos de barcos, tienen que haber sido guardados en un fichero barcos.txt
3. No poseen animaciones gráficas
4. No existe un sistema de puntuaciones que premie a quien dispara más rápido, ni existe una función que de todas las partidas jugadas, diga cual ha sido la mejor estadística de aciertos en proporción al agua.

5. El juego no posee una IA o una cpu con la que puedas jugar, y tenga una dificultad que según ella juegue peor o mejor.

- Ejemplos previos del funcionamiento del juego: Aquí se podran observar algunos de los ejemplos a tener en cuenta del juego.

Figura 1: Configuraciones no válidas para los barcos.



Figura 2: Colocación de barcos de forma válida.

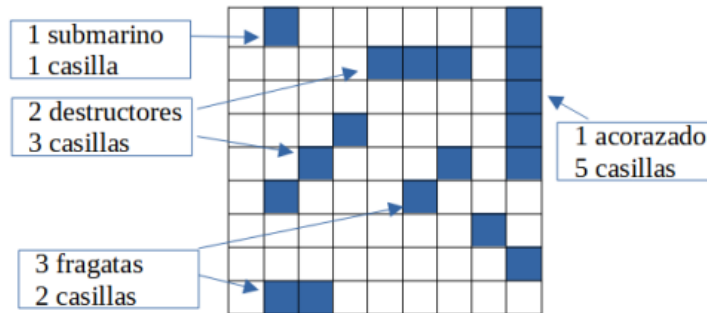
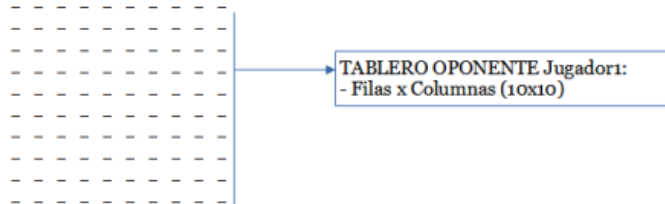


Figura 3: Tablero del oponente ejemplo.



2.2. Tecnología

Bien, en cuanto el número de tecnologías empleadas en el proyecto podemos destacar: **codeblock**, **github**, o las variadas **librerías** creadas a partir de codeblock, inteligencias artificiales (nose si ponerlas).

Para empezar, **codeblock** es nuestro compilador fundamental que hemos utilizados para realizar todas las pruebas de caja blanca y caja negra, asegurandonos que todos los modulos y codigo funcionaban correctamente, la sintaxis del lenguaje sea correcta, y que cumplieran sus objetivos y no de errores. Por otro lado las **librerías** han sido una parte importante, ya que dentro del compilador ha sido la clave para que todos los modulos se cohesionasen entre sí, también tenemos **github** imprescindible para poder interactuar entre los miembros del grupo y poder realizar los cambios de codigo, sin tener la necesidad de estar guardando constantemente, descargarlo y tener que organizar todo para probar desde el main que funciona correctamente.

2.3. Manual de instalación

Tendrás que seguir detalladamente las siguientes especificaciones:

- Descarga e Instalación de CodeBlocks
 - Requisitos Previos: Disponer de un sistema Operativo por ejemplo Windows 7/10/11. Y un espacio en el disco duro: 500 MB libres.
 - Pasos para descargar CodeBlocks :Primero acceder a la web oficial en el enlace <http://www.codeblocks.org/downloads>, buscar el instalador de la versión adecuada, codeblocks-20.03mingw-setup.exe. Una vez descargado el archivo, seguir los pasos del asistente y MinGW como compilador predeterminado.
- Configuración Inicial de Code::Blocks
 - Configurar el Compilador: Primero abrimos la aplicación CodeBlocks, clicar en Settings entrando en COMPILER, una vez seleccionado el compilador, elige GNU GCC Compiler. Luego haz click en Toolchain executables y verifica que la ruta de gcc.exe sea correcta, por ejemplo(C:).
 - Verificamos la instalación sea correcta, creamos un proyecto de prueba siguiendo los pasos, primeros pinchamos en File, luego elegimos la opción New, nos vamos a Project y por último buscamos la opción Console application, donde compilando y confirmando se nos guarda todo.
- Descargar el código fuente: descargamos los archivos(del zip o donde sea que se vaya a mandar), y una vez guardados en nuestro nuevo proyecto

creado, desde project view, clickeamos en Add files y agregamos todos los archivos .c y .h .

- Comprobación y juego: Una vez todo el código fuente este preparado, preparamos los ficheros necesarios para jugar, uno que se llame barcos.txt donde se almacenan tus tipos de barcos con los que quieres jugar y otro juego.txt donde se guardara y se ira sobrescribiendo las partidas. Una vez hecho esto probamos en build (la rueda) que no salte ni un warning ni un error, por último compilamos y a jugar!!

2.4. Acceso al sistema

Nuestro juego, hundir la flota solo necesitará de una vez realizado correctamente todo el proceso de instalación, de realizar ahora los siguientes pasos:

- Inicialización y desarrollo del juego
 - Iniciar codeblock tras previamente haber seguido el manual de instalación correctamente.
 - Cargar el proyecto una vez descargado previamente en el proceso de instalación.
 - Asegurarse que los ficheros esten bien guardados y que cumplan con las diferentes especificaciones dichas.
 - Proceder a compilar y jugar al juego
 - Cuando se desee desde "Menu Principal", darle a la opcion 3: "3. Salir".
- Cosas a tener en cuenta:
 - Los barcos no se pueden colocar de cualquier manera, hay que seguir reglas que se aclaran durante el juego.
 - No introducir un tablero con un tamaño menor a 3
 - En los disparos se castiga si disparas a una misma zona de nuevo, perdiendo el turno consecuentemente.

2.5. Manual de referencia y guía del operador

El desarrollo del juego constara principalmente en 3 partes, la primera en configurar la partida, luego la segunda en jugar la partida, y por último salir de la partida, estas será imprescindible a tomar en cuenta para llevar a cabo el desarrollo del juego:

- Configurar partida: Será la primera parte y será imprescindible para poder jugar, así que esta debe de ser aplicada al comienzo, ya que si la configuración no se encuentra guardada, no se podrá inicializar el juego, tendremos que desde el menu principal pulsar "1".
 - Introducir datos: Esta función sera clave tras recibir los tipos de barcos a escoger se guardará el número de barcos y sus distintos tipos con los que se va a jugar, se pedirá con que tipo de disparo se va a jugar bien manual o automático y a quien se le va a atribuir el primer turno. También se pedira un nombre para ir identificando al usuario.
 - Mostrar datos: Bien, mostrar datos nos asegurará que la configuración de barcos y de jugadores ha sido correcta.
 - Borrar datos: Borrar datos nos ayudará una vez haber visto los datos en mostrar datos no estamos satisfecho, o tras jugar queremos una nueva configuración pues borrar datos reseteara todos los datos.
 - Guardar datos: Guardar datos fundamental para asegurarnos que todos los datos han sido almacenados, siempre al introducir datos hay que pasar por este paso.
 - Cargar datos: Una función útil si hemos cerrado la terminal, ya que si hemos guardado correctamente la partida, podremos cargar los datos de una partida previamente jugada.
 - Volver al menu principal: Simplemente, nos llevara de vuelta al menu principal.
- Jugar partida: Jugar la partida es una opción que nos llevara al propio algoritmo del juego, basicamente jugar al "hundir la flota", para ello se encontrará con las distintas posibilidades. Desde el menu principal pulsando la opción "2", nos desenvolveremos en el juego.
 - Desarrollo Partida: Iniciar la partida, tras haber guardado todos los datos, si no existen datos no cargara correctamente, y se desarrollará de la siguiente forma:
 1. Colocar la flota: Se tendra que escoger o bien de forma automatica o manual el modo de colocación de los barcos, respetando las normas básicas de hundir la flota, donde no se puede colocar fuera un barco fuera de los limites de tablero, y hay que tener en cuenta que los barcos no se pueden tocar, tiene que haber al menos una

casilla vacía entre ellos.

2. Acaba con el enemigo: En este momento estarás jugando directamente, teniendo que disparar a los barcos de la flota enemigo, según el tipo disparo configurado previamente, se realizaran disparos, si es tocado o hundido el juego le dara un turno extra cuantas veces acierte el disparo, si falla automaticamente pasa a jugar el otro jugador. A lo largo de esta etapa te podra pedir dos opciones, bien seguir disparando, o salir de la partida.

3. Ganador: El jugador que hunda la flota primero se proclamará como el ganador de la partida, teniendo un ganador y un perdedor acabando la partida.

- Reiniciar Partida: Si en mitad de la partida has deseado salir de la partida, bien para dejar de jugar, o querer reiniciar la partida con esta opción reseteara toda la partida, teniendo los datos ya guardados.
 - Resumen de partida: Es una función muy interesante para conocer todas las estadísticas realizadas a lo largo de la partida, como el número de disparos, o el número de casillas o bien tocadas, agua, hundidas y vacías, o el número de barcos hundidos.
 - Volver al menu principal: Mismo propósito del de la función volver del menu de configuración.
- Salir partida: Para salir de la terminal donde hemos estado jugando el juego tendremos que elegir esta opción, desde el menu principal pulsaremos "3." eligiendo esta opción para salir del juego.

3. Documentación de sistema

Documentación referida a los aspectos del análisis, diseño, implementación y prueba del *software*, así como la implantación del mismo. En general, debe hacer referencia al sistema, ya que está orientada a los programadores que van a realizar el mantenimiento. Debe estar muy bien estructurada, desde lo más general a los más interno.

3.1. Especificación del sistema

- Análisis del juego: Hundir la flota, objetivo hundir los barcos del oponente, los componentes fundamentales del juego, como la disposición de los barcos, dos tableros por jugador, un tablero su flota, y otro tablero de su oponente, donde se realiza los disparos. Sistema funcionara por turnos, dependiendo del disparo que se realice.
- Las especificaciones que debe de seguir el software para que cumpla los requisitos:
 - El sistema debe permitir colocar barcos en un tablero.
 - El jugador y la IA deben alternar turnos para atacar coordenadas.
 - El sistema debe comprobar si un disparo es .^aagua”, .^oimpacto.” o ”hundido”.
 - Mostrar el estado del tablero (enemigo) después de cada turno.
 - Determinar automáticamente el fin del juego e indicar el ganador (cuando todos los barcos se hunden).
 - El sistema de colocación de barcos sea correcto.
 - Cumplir algoritmo disparo automático.
 - Los tipos de barcos (tamaño, id, nombre barco) almacenado de forma correcta en un fichero barco.txt
 - El sistema puede bien hacer un resumen de una partida, borrarla y cargarla una vez pausada
- Para el juego hundir la flota hemos hecho la siguiente descomposición para realizar el desarrollo: Para el juego hundir la flota hemos hecho la siguiente descomposicion para realizar el desarrollo: el main.c que es el eje gestor de todo el resto de funciones, que van a encontradas y almacenadas en distintos modulos. En total hemos usado hasta 6 modulos distintos los cuales abarcaran un rango de funciones: primero y

la más importante el modulo interfaces” que se encargara de llevar a cabo el desarrollo del juego gracias a las distintas funciones que posee y la aportación del resto de modulos. También tenemos el modulo configuración” la cual se encargara de la configuración básica e inicial del juego, más modulos como colocar flota” que recibiendo los tipos de barcos en un fichero se encargará de colocar estos en un tablero (creado en configuración), ”disparoAutomatico” un algoritmo que disparará automáticamente, no de manera aleatoria, sino con cierta lógica y se asociara con el modulo resultadoDisparo”, donde según el disparo que reciba del jugador devolverá un valor a la interfaz, y por último pero no menos importante un último modulo ”pospartida” que imprimira toda las estadísticas producidas a lo largo de la partida.

- Plan desarrollo del *software*.

3.2. Módulos

Los diferentes modulos serían:

- Main.c: este como su nombre lo dice es el ”main”, es decir, el modulo principal y gestor del resto de modulos. Su papel es fundamental, para poder compilar lo que es el juego.
- Interfaces.c/.h: imprescindible, ya que se encargara de llevar a cabo el desarrollo del juego gracias a las distintas funciones que posee y ya que recibe todas las llamadas del resto de modulos, este junto a otros modulos se encargaran de enviar todos los datos al main. Interfaces es fundamental ya que realizará el proceso de interfaz entre el resto de módulos. Desde el menú principal se podrá realizar tres opciones, o bien configurar la partida, jugar la partida, y salir del juego. Por último en jugar partida se encargara de mediante llamadas a otros modulos y funciones propias asegurarse del funcionamiento del desarrollo de jugar partida, de esta manera este módulo llevará a cabo el sistema de turnos y actualizando el tablero a lo largo de la partida.
- Configuracion.c/.h: Se encargará de procesar la parte del juego que se encarga de configurar la partida, pidiendonos introducir datos como el tamaño del tablero con el que se quiere jugar, el tipo de disparo que desea cada jugador (manual o automático), la asignación de turno de forma automática o manual, la posibilidad de o bien borrar partida, guardar partida, y cargar partida. También muy importante recogerá

todos los tipos de barcos que se pueden pedir para poder jugar la partida, Por último también es capaz de recopilar el números de barcos hundidos por un jugador.

- ColocarFlota.c/.h: Tras recibir todos los datos una vez configurados se encargará de preparar los tableros de los jugadores, pidiendo los tipos de barcos que se desean, el número de ellos, el modo de colocación respetando las normas si deseas que se coloquen de forma manual o automática. Una vez guardado, y los tableros preparados, todo se enviará a la interfaz para que el juego se pueda comenzar
- Disparoautomatico.c/.h: un algoritmo importante, para la lógica del juego, ya que en caso que el usuario haya deseado elegir el disparo automático, disparará con cierta lógica donde a través de la asociación de otro modulo (resultadoDisparo) este tipo de disparo realizara donde segun el disparo que realice el jugador, si encuentra un barco (tamaño superior a 1) buscará en los alrededores (posiciones cardinales y adyacentes), si fuera un barco de tamaño mayor a 3, si vuelve acertar, según la posición elegirá o bien la proa o la popa del barco para terminar de hundirlo, y hablaremos del módulo que se relaciona directamente con este.
- Disparoautomatico.c/.h: un algoritmo importante, para la lógica del juego, ya que en caso que el usuario haya deseado elegir el disparo automático, disparará con cierta lógica donde a través de la asociación de otro modulo (resultadoDisparo) este tipo de disparo realizara donde segun el disparo que realice el jugador, si daña a un barco (tamaño superior a 1) buscará en los alrededores (posiciones cardinales y adyacentes),ahora si fuera un barco de tamaño mayor a 3,y si vuelve acertar, según la posición elegirá o bien la proa o la popa del barco (según la orientación para un lado o otro), (¡importante ,no se podrá confundir con los límites del tablero la IA!) , para terminar de hundirlo, y hablaremos del módulo que se relaciona directamente con este.
- ResultadoDisparo.c/.h: Según el disparo que se registre en el tablero del jugador contrario devolvera un valor a la interfaz, siendo o bien tocado, agua, hundido o repetido. Este módulo devolvera valores al disparo automático si el jugador ha elegido el disparo automático.(proceso explicado en su debido modulo).
- Pospartida.c/.h: último pero no menos importante un último modulo "pospartida" que imprimira toda las estadísticas producidas a lo largo de la partida.

3.3. Plan de prueba

Debe describir todo el plan de prueba que se ha llevado a cabo. Se distinguen las pruebas realizadas a cada módulo y las pruebas de integración de los distintos módulos probándolo como un todo. Por último, también se debe incluir la descripción del plan de pruebas de aceptación.

Prueba de los módulos

A continuación se describen las pruebas realizadas sobre los diferentes módulos del sistema, en especial sobre las funciones interactivas que requieren validación de entrada por parte del usuario. Se detallan pruebas de caja negra y de caja blanca.

Función: `obtenerOpcion(int min, int max)` Esta función solicita al usuario una opción dentro de un rango definido. La entrada debe ser numérica y dentro del intervalo establecido por los argumentos `min` y `max`.

De esta función se ha encargado de hacer las pruebas José Gómez García.

```
1  int obtenerOpcion(int min, int max) {
2      int opcion;
3      int valido = 0;
4      //Nodo 1: entrada a funcion y declaracion de variables
5      while (!valido) {
6          //Nodo 2: entrada al bucle
7          printf("\nTu opcion: ");
8          if (scanf("%d", &opcion) != 1) {
9              //Nodo 3: entrada no numerica
10             printf("\nEntrada invalida, debes introducir un
11                 numero\n");
12             } else if (opcion >= min && opcion <= max) {
13                 //Nodo 4: entrada valida y dentro del rango
14                 valido = 1;
15             } else {
16                 //Nodo 5: entrada valida pero fuera del rango
17                 printf("\nNumero no valido, introduce una opcion
18                     entre %d y %d.\n", min, max);
19             }
20             limpiarBuffer();//Nodo 6: ejecucion de dicha funcion
21     }
22     return opcion;//Nodo 7: salida
23 }
```

Listing 1: Función `obtenerOpcion`

Pruebas de Caja Blanca: En la prueba de caja blanca se han identificado los siguientes nodos del flujo de control:

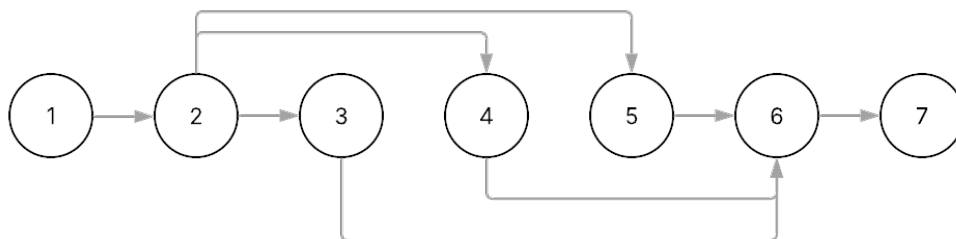
- Nodo 1: Entrada a función y declaración de variables
- Nodo 2: Entrada al bucle
- Nodo 3: Entrada no numérica
- Nodo 4: Entrada válida dentro de rango
- Nodo 5: Entrada válida fuera del rango
- Nodo 6: Limpieza del búfer
- Nodo 7: Salida

Se han recorrido los siguientes caminos:

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 7$: Entrada no válida como texto (ej. abc)
- $1 \rightarrow 2 \rightarrow 5 \rightarrow 6 \rightarrow 7$: Número fuera del rango
- $1 \rightarrow 2 \rightarrow 6 \rightarrow 7$: Entrada válida

Aquí adjuntamos un grafo que representa los posibles caminos:

Figura 4:



Estas pruebas garantizan que la función maneja correctamente entradas inválidas, tanto por tipo como por rango, y repite la solicitud hasta obtener una entrada correcta.

Pruebas de caja negra: Las pruebas de caja negra consideran los posibles valores de entrada sin observar la implementación interna.

Caso	Entrada	Rango válido (1-3)	Resultado esperado
1	2	1 - 3	Retorna 2
2	0	1 - 3	Error, pide nuevo valor
3	abc	1 - 3	Error, pide nuevo valor
4	4	1 - 3	Error, pide nuevo valor
5	3	1 - 3	Retorna 3

Cuadro 1: Pruebas de caja negra de la función `obtenerOpcion`

Análisis del Grafo de Control de Flujo (CFG): El grafo de control de flujo correspondiente a la función `obtenerOpcion` contiene los siguientes nodos:

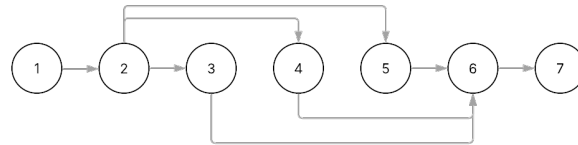


Figura 5: Grafo de Control de Flujo de `obtenerOpcion`

Cálculo de la Complejidad Ciclomática: La complejidad ciclomática ($V(G)$) se calcula mediante tres métodos:

■ **Fórmula de aristas y nodos:**

$$V(G) = NA - NN + 2 = 8 - 7 + 2 = 3$$

(Donde $NA = 8$ aristas y $NN = 7$ nodos)

■ **Nodos predicado:**

$$V(G) = NNP + 1 = 2 + 1 = 3$$

(Nodos predicado: 3 y 5)

■ **Regiones:**

$$V(G) = 3regiones(exterior + 2ciclos)$$

Interpretación: La complejidad ciclomática $V(G) = 3$ indica:

- 3 caminos linealmente independientes
- Número mínimo de casos de prueba para cobertura de caminos
- Umbral de mantenibilidad (valores menores de 10 son críticos)

Los caminos identificados coinciden con los casos de prueba ejecutados:

1. Camino 1-2-3-6-2 (entrada no numérica)
2. Camino 1-2-4-6-2 (entrada válida)
3. Camino 1-2-5-6-2 (entrada fuera de rango)

Este análisis confirma que las pruebas realizadas cubren el 100 % de los caminos básicos del CFG.

Función: leerBarcos(int *nBarcosDisponibles) Esta función se encarga de leer una información extraída de un fichero, concretamente los datos relacionados con los barcos, su nombre, id y tamaño. Además, se encarga de guardarlo en una estructura.

De esta función se ha encargado de hacer las pruebas Francisco Rodriguez Alcón.

```
1 static barcos *leerBarcos(int *nBarcosDisponibles){
2 //Nodo 1: entrada a funcion y declaracion de variables
3   barcos *barcosDisponibles;
4   FILE *fBarcos;
5   int indexBarcosDisponibles = 0;
6   char infoBarcos[50]; //Informacion a desglosar
7
8   fBarcos = fopen(CONFIG_BARCOS, "r"); //Nodo 2
9
10  if(fBarcos == NULL){ //Nodo 3
11    printf("No se pudo abrir el fichero de configuracion de
12          barcos. \n");
13    exit(1); //Nodo 4
14  }
15
16  *nBarcosDisponibles = numLineasFichero(fBarcos); //Nodo 5
17
18  barcosDisponibles = (barcos *)malloc(*nBarcosDisponibles *
19    sizeof(barcos)); //Nodo 6
```

```

18  if(barcosDisponibles == NULL){//Nodo 7
19      printf("No se pudo reservar memoria para los barcos
20             jugables. \n");
21      exit(1); //Nodo 8
22  }
23  while(!feof(fBarcos)){ //Nodo 9
24      fgets(infoBarcos, 50, fBarcos); //Nodo 10
25      desglosarInfoBarcos(infoBarcos, barcosDisponibles+
26                          indexBarcosDisponibles); //Nodo 10
27      indexBarcosDisponibles++; //Nodo 10
28  }
29  fclose(fBarcos); //Nodo 11
30  return barcosDisponibles; //Nodo 12
31  }

```

Listing 2: Función leerBarcos

Pruebas de Caja Blanca: En la prueba de caja blanca se han identificado los siguientes nodos del flujo de control:

- Nodo 1: Entrada a función y declaración de variables
- Nodo 2: Apertura del archivo
- Nodo 3: Verificación de apertura
- Nodo 4: Salida por error de apertura
- Nodo 5: Cálculo de número de líneas
- Nodo 6: Reserva de memoria
- Nodo 7: Verificación de memoria
- Nodo 8: Salida por error de memoria
- Nodo 9: Condición del bucle while
- Nodo 10: Lectura y procesamiento de línea
- Nodo 11: Cierre del archivo
- Nodo 12: Retorno del resultado

Se han recorrido los siguientes caminos:

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$: Error al abrir archivo
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$: Error al reservar memoria
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 10 \rightarrow 9 \rightarrow 11 \rightarrow 12$: Flujo normal con 1 línea
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 10 \rightarrow 9 \rightarrow 10 \rightarrow 9 \rightarrow 11 \rightarrow 12$: Flujo normal con 2 líneas

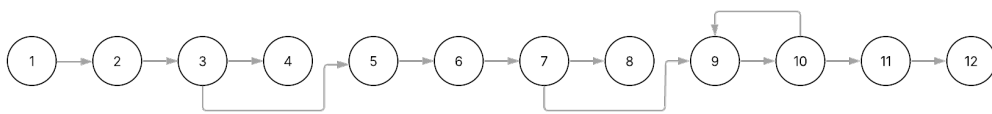


Figura 6: Grafo de Control de Flujo de `leerBarcos`

Pruebas de caja negra: Las pruebas de caja negra consideran los posibles valores de entrada sin observar la implementación interna.

Caso	Entrada	Resultado esperado
1	Archivo no existe	Mensaje error y <code>exit(1)</code>
2	Archivo vacío	<code>*nBarcos=0</code> , retorna array vacío
3	Archivo con 1 línea válida	<code>*nBarcos=1</code> , retorna 1 barco
4	Archivo con 3 líneas válidas	<code>*nBarcos=3</code> , retorna 3 barcos
5	Sin memoria disponible	Mensaje error y <code>exit(1)</code>

Cuadro 2: Pruebas de caja negra de la función `leerBarcos`

Análisis del Grafo de Control de Flujo (CFG): El grafo de control de flujo correspondiente a la función `leerBarcos` contiene 12 nodos.

Cálculo de la Complejidad Ciclomática:

- **Fórmula de aristas y nodos:**

$$V(G) = NA - NN + 2 = 14 - 12 + 2 = 4$$

(12 nodos, 14 aristas)

- **Nodos predicado:**

$$V(G) = NNP + 1 = 3 + 1 = 4$$

(Nodos predicado: 3, 7 y 9)

- **Regiones:**

$$V(G) = 4regiones$$

Interpretación: La complejidad ciclomática $V(G) = 4$ indica:

- 4 caminos linealmente independientes
- Se requieren 4 casos de prueba para cobertura completa
- Complejidad moderada (umbral crítico ≤ 10)

Los caminos identificados coinciden con los casos de prueba:

1. Error apertura archivo
2. Error reserva memoria
3. Flujo normal 1 línea
4. Flujo normal múltiples líneas

Este análisis confirma que las pruebas cubren el 100 % de los caminos básicos.

Función: `resultado resultadoDisparo(int fila, int columna, int tam_tablero, char **tableroFlota, char **tableroOponente)` Esta función devuelve un valor enumerado del resultado, TOCADO, HUNDIDO, AGUA, REPETIDO, dependiendo de donde se realice el disparo en la flota del oponente.

De esta función se ha encargado de hacer las pruebas Alfonso Pacheco Marti.

```

1 resultado resultadoDisparo(int fila, int columna, int
  tam_tablero, char **tableroFlota, char **tableroOponente)
  {
2     // Reservamos matriz de visitado, Nodo 1: reservamos
      memoria en matrices auxiliares
3     int **auxVisitado = malloc(tam_tablero * sizeof(int *));
4     int **coordenadas = malloc(tam_tablero * tam_tablero *
      sizeof(int *));
5     //Nodo 2:
6     for (int i = 0; i < tam_tablero; i++) {

```

```

7         auxVisitado[i] = calloc(tam_tablero, sizeof(int));
8     }
9     //Nodo 3:
10    for (int i = 0; i < tam_tablero * tam_tablero; i++) {
11        coordenadas[i] = malloc(2 * sizeof(int)); // cada
            posicion guarda [fila, columna]
12    }
13    //Nodo 4:
14    if(tableroOponente[fila][columna] == 'A' ||
        tableroOponente[fila][columna] == 'H' ||
        tableroOponente[fila][columna] == 'T'){
15        // Nodo 11:
16        return REPETIDO;
17    }
18    //Nodo 5:
19    // Comprobamos el disparo
20    if (tableroFlota[fila][columna] == 'X') {
21        tableroOponente[fila][columna] = 'T';
22
23        int contador = 0;
24        // Nodo 6:
25        int hundido = buscarBarco(fila, columna, tam_tablero,
            tableroFlota, tableroOponente, auxVisitado,
            coordenadas, &contador);
26
27        /// Nodo 7:
28        if (hundido) {
29            // Nodo 8:
30            for (int i = 0; i < contador; i++) {
31                int x = coordenadas[i][0];
32                int y = coordenadas[i][1];
33                tableroOponente[x][y] = 'H';
34            }
35
36            // insertar aqui: marcar adyacentes como agua
37            int dx[] = {-1, -1, -1, 0, 0, 1, 1, 1};
38            int dy[] = {-1, 0, 1, -1, 1, -1, 0, 1};
39            // Nodo 9:
40            for (int i = 0; i < contador; i++) {
41                int x = coordenadas[i][0];
42                int y = coordenadas[i][1];
43                for (int k = 0; k < 8; k++) {
44                    int nx = x + dx[k];
45                    int ny = y + dy[k];
46
47                    if (nx >= 0 && nx < tam_tablero && ny >=
                        0 && ny < tam_tablero) {
48                        if (tableroOponente[nx][ny] == '-') {
49                            tableroOponente[nx][ny] = 'A';

```

```

50         }
51     }
52 }
53 }
54 // Nodo 10:
55
56 // Liberar memoria antes de retornar
57 for (int i = 0; i < tam_tablero; i++) free(
    auxVisitado[i]);
58 free(auxVisitado);
59 for (int i = 0; i < tam_tablero * tam_tablero; i
    ++) free(coordenadas[i]);
60 free(coordenadas);
61 // Nodo 11:
62 return HUNDIDO;
63 } else {
64     // Liberar memoria antes de retornar
65     for (int i = 0; i < tam_tablero; i++) free(
        auxVisitado[i]);
66     free(auxVisitado);
67     for (int i = 0; i < tam_tablero * tam_tablero; i
        ++) free(coordenadas[i]);
68     free(coordenadas);
69     // Nodo 11:
70     return TOCADO;
71 }
72 } else {
73     tableroOponente[fila][columna] = 'A';
74
75     // Liberar memoria antes de retornar
76     for (int i = 0; i < tam_tablero; i++) free(
        auxVisitado[i]);
77     free(auxVisitado);
78     for (int i = 0; i < tam_tablero * tam_tablero; i++)
        free(coordenadas[i]);
79     free(coordenadas);
80     // Nodo 11:
81     return AGUA;
82 }
83
84 }

```

Listing 3: Función resultadoDisparo

Pruebas de Caja Blanca: En la prueba de caja blanca se han identificado los siguientes nodos del flujo de control:

- Nodo 1: Reserva de memoria para matrices auxiliares

- Nodo 2: Verificación de disparo repetido
- Nodo 3: Comprobación de impacto en barco ('X')
- Nodo 4: Procesamiento de disparo en agua
- Nodo 5: Procesamiento de disparo en barco
- Nodo 6: Llamada recursiva a `buscarBarco`
- Nodo 7: Decisión de barco hundido
- Nodo 8: Marcado completo de barco hundido
- Nodo 9: Marcado de celdas adyacentes
- Nodo 10: Liberación de memoria
- Nodo 11: Puntos de retorno

Se han recorrido los siguientes caminos:

- $1 \rightarrow 2 \rightarrow 11$: Disparo repetido (retorno REPETIDO)
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 10 \rightarrow 11$: Disparo en agua (retorna AGUA)
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 10 \rightarrow 11$: Disparo que toca barco (retorna TOCADO)
- $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11$: Disparo que hunde barco (retorna HUNDIDO)

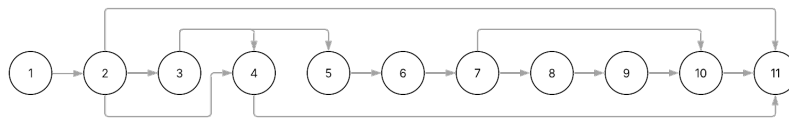


Figura 7: Grafo de Control de Flujo de `resultadoDisparo`

Pruebas de caja negra:

Caso	Coordenadas	Estado inicial	Resultado esperado	Descripción
1	(0,0)	Á'(agua previa)	REPETIDO	Disparo repetido
2	(2,3)	'-'(vacía)	AGUA	Disparo en agua
3	(1,1)	'X'(barco)	TOCADO	Impacto en barco no hundido
4	(4,4)	'X'(última pieza)	HUNDIDO	Disparo que hunde barco
5	(9,9)	'X'(borde)	TOCADO/HUNDIDO	Validación de límites

Cuadro 3: Pruebas de caja negra de la función `resultadoDisparo`

Cálculo de la Complejidad Ciclomática:

- **Fórmula de aristas y nodos:**

$$V(G) = NA - NN + 2 = 15 - 11 + 2 = 6$$

- **Nodos predicado:**

$$V(G) = NNP + 1 = 5 + 1 = 6$$

(Nodos predicado: 2, 3, 5, 7, 9)

- **Regiones:**

$$V(G) = 6regiones(exterior + 5decisiones)$$

Interpretación:

- 6 caminos linealmente independientes
- 6 casos de prueba necesarios para cobertura completa
- Umbral de mantenibilidad aceptable (¡10)

Los caminos de prueba cubren:

1. Disparo repetido
2. Disparo en agua
3. Impacto no hundido
4. Barco hundido
5. Validación de bordes
6. Marcado de adyacentes

Función: void disparoAutomatico(char **tablero_oponente, char**tablero_flota, int*tam_tablero, int*fila, int*columna, int*ultima_fila, int*ultima_columna, int*numTocado, estado*estadoDisparoAnterior, int*ultima_direccionX, int*ultima_direccionY, resultado*resultadoDisparoA) Esta función se encarga de .

De esta función se ha encargado de hacer las pruebas Daniel González Vega.

```
1 void disparoAutomatico(  
2     char **tablero_oponente,  
3     char **tablero_flota,  
4     int tam_tablero,  
5     int *fila, int *columna,  
6     int *ultima_fila, int *ultima_columna,  
7     int *numTocado,  
8     estado *estadoDisparoAnterior,  
9     int *ultima_direccionX, int *ultima_direccionY,  
10    resultado *resultadoDisparoA  
11 ){  
12  
13 }
```

Listing 4: Función disparoAutomatico

Pruebas de Caja Blanca: En la prueba de caja blanca se han identificado los siguientes nodos del flujo de control:

- Nodo 1: Entrada a función y declaración de variables
- Nodo 2: Estado ALEATORIO: se genera coordenada aleatoria
- Nodo 3: Disparo aleatorio da TOCADO → cambia a ALREDEDOR
- Nodo 4: Estado ALREDEDOR: se recorre el vecindario
- Nodo 5: Disparo alrededor da TOCADO → cambia a DIRECCION
- Nodo 6: Disparo alrededor da HUNDIDO → cambia a ALEATORIO
- Nodo 7: No se encuentra dirección válida → vuelve a ALEATORIO
- Nodo 8: Estado DIRECCION: se dispara en dirección registrada
- Nodo 9: Disparo da TOCADO → se continúa
- Nodo 10: Disparo da HUNDIDO → cambia a ALEATORIO

- Nodo 11: Disparo da AGUA $\rightarrow$ cambia a CAMBIO_DIRECCION
- Nodo 12: Estado CAMBIO_DIRECCION: dispara en dirección opuesta
- Nodo 13: Da TOCADO $\rightarrow$ vuelve a DIRECCION
- Nodo 14: Da HUNDIDO o AGUA $\rightarrow$ cambia a ALEATORIO
- Nodo 15: Salida

Caminos recorridos:

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 15$: Disparo aleatorio da TOCADO (pasa a ALREDEDOR)
- $1 \rightarrow 2 \rightarrow 15$: Disparo aleatorio da AGUA (permanece en ALEATORIO)
- $1 \rightarrow 4 \rightarrow 5 \rightarrow 15$: Disparo alrededor da TOCADO (pasa a DIRECCION)
- $1 \rightarrow 4 \rightarrow 6 \rightarrow 15$: Disparo alrededor da HUNDIDO (pasa a ALEATORIO)
- $1 \rightarrow 4 \rightarrow 7 \rightarrow 15$: Ningún disparo válido en ALREDEDOR (pasa a ALEATORIO)
- $1 \rightarrow 8 \rightarrow 9 \rightarrow 15$: Disparo en DIRECCION da TOCADO (se mantiene)
- $1 \rightarrow 8 \rightarrow 10 \rightarrow 15$: Disparo en DIRECCION da HUNDIDO (vuelve a ALEATORIO)
- $1 \rightarrow 8 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow 15$: Dirección original falla, opuesta da TOCADO
- $1 \rightarrow 8 \rightarrow 11 \rightarrow 12 \rightarrow 14 \rightarrow 15$: Ambas direcciones fallan

Pruebas de caja negra: Las pruebas de caja negra consideran los posibles valores de entrada sin observar la implementación interna. Se evalúan las respuestas del sistema según el estado del disparo anterior y el contenido de la celda objetivo.

Caso	Estado anterior	Resultado esperado	Observación
1	ALEATORIO	TOCADO	Cambia estado a ALREDEDOR
2	ALEATORIO	AGUA	Permanece en ALEATORIO
3	ALREDEDOR	TOCADO	Cambia estado a DIRECCION
4	ALREDEDOR	HUNDIDO	Cambia estado a ALEATORIO
5	ALREDEDOR	N/A	Salta al siguiente
6	DIRECCION	TOCADO	Se mantiene en DIRECCION y avanza
7	DIRECCION	AGUA	Cambia a CAMBIO_DIRECCION
8	CAMBIO_DIRECCION	TOCADO	Vuelve a estado DIRECCION
9	CAMBIO_DIRECCION	AGUA	Cambia a ALEATORIO
10	CAMBIO_DIRECCION	N/A	Cambia a ALEATORIO por salto

Cuadro 4: Pruebas de caja negra de la función `disparoAutomatico`

Cálculo de la Complejidad Ciclomática: La complejidad ciclomática ($V(G)$) se calcula de las siguientes formas:

■ **Fórmula de aristas y nodos:**

$$V(G) = NA - NN + 2 = 19 - 17 + 2 = 4$$

■ **Nodos predicado:**

$$V(G) = NNP + 1 = 3 + 1 = 4$$

(Nodos predicado: ALEATORIO, ALREDEDOR, DIRECCION)

■ **Regiones:**

$$V(G) = 4regiones$$

Interpretación: La complejidad ciclomática $V(G) = 4$ indica:

- Se requieren al menos 4 pruebas para cubrir todos los caminos básicos
- La función tiene una estructura compleja pero mantenible (valor menor a 10)

Los caminos identificados coinciden con los casos clave de prueba:

1. Camino 1-2-3-15: Disparo aleatorio da TOCADO
2. Camino 1-4-5-15: Disparo alrededor da TOCADO

3. Camino 1-8-11-12-13-15: Dirección inicial falla, opuesta da TOCADO

4. Camino 1-8-11-12-14-15: Ambas direcciones fallan

Estas pruebas garantizan la cobertura del 100 % de los caminos independientes del grafo de control.

Interpretación: Estas pruebas validan los distintos caminos y estados posibles en función del resultado del disparo:

- Se contemplan transiciones entre los 4 estados posibles: ALEATORIO, ALREDEDOR, DIRECCION, CAMBIO_DIRECCION
- Se verifica el comportamiento ante disparos válidos, repetidos, fuera del tablero o cuando un barco es hundido
- Garantiza que el comportamiento del disparo automático se adapta correctamente al contexto del juego

Plan de pruebas de aceptación

Estas pruebas deben diseñarse con el usuario del sistema. Deben describir las pruebas que son necesarias pasar para que el sistema sea aceptado por el usuario final.

Si en algún punto del documento se quiere hacer referencia a algún documento es necesario incluir la cita donde corresponda, podemos tomar como ejemplo ?.

4. Documentación del código fuente

En este punto se puede incluir toda la documentación interna que puede ser generada con Doxygen.

La documentación interna del programa, como se ha comentado anteriormente, comprende los comentarios del programa. Durante la fase de implementación es necesario comentar adecuadamente cada una de las partes del programa. Estos comentarios se incluyen en el código fuente con el objetivo de aclarar qué es lo que hace el fragmento de código, para explicar sus elementos.

Esta documentación ayuda a hacer más comprensible el código fuente a los programadores que deban trabajar en él, facilitando las tareas de mantenimiento. No sólo proporciona una mayor legibilidad en la actualización y modificación del código, sino también en la depuración del mismo.

Todas las cabeceras, precondiciones y postcondiciones se encuentran en los propios módulos redactados, así que en la documentación no se encuentra nada.